ROAMIST

Pakistan Travel App — Android Native

LOGIC MAP

Implementation Flow & Requirement Traceability Document

App Name: Roamist — Pakistan Travel Companion

Platform: Android Native (Kotlin)

Architecture: Fragment-based Modular UI with Intent/Bundle Communication

Date: March 2026

Version: v1.0

LOGIC_MAP.PDF

1 Project Architecture Overview

📁 Package Structure

- **activities/** — 16 Activity classes
- **fragments/** — 5 Fragment classes
- **adapters/** — DestinationAdapter
- **models/** — Destination (Parcelable)
- **res/layout/** — 13+ XML layouts

🔧 Technology Stack

- Language: Kotlin
- Min SDK: API 21 (Android 5.0+)
- UI: Material Design Components
- Lists: RecyclerView + LinearLayoutManager
- Navigation: Intent Extras + Bundle

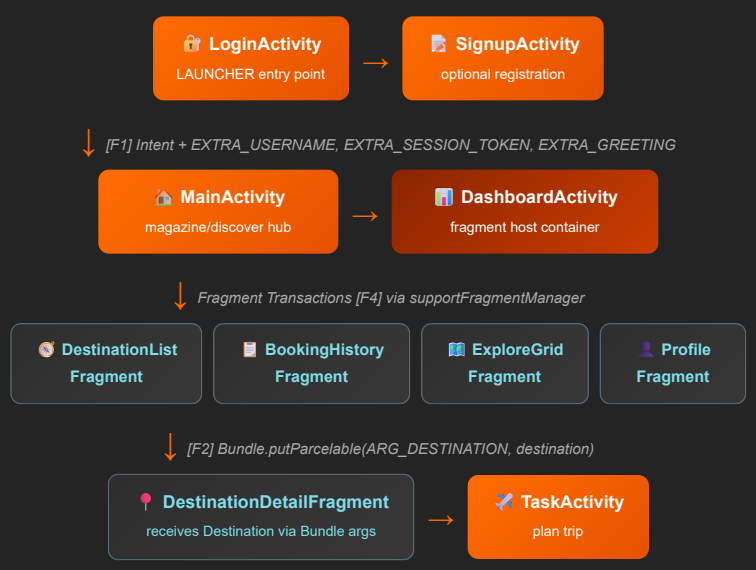
🎨 Design Tokens

- Brand: Pakistan Green #01411C
- Accent: Orange #FF6D00
- Theme: Dark Mode (custom Roamist)
- Currency: PKR (Pakistani Rupee)
- Content: 12 real Pakistan destinations

✅ Functional Requirements

- **F1** — Intent Navigation (Login → Dashboard)
- **F2** — Bundle Transfer (Destination Parcelable)
- **F3** — RecyclerView with custom Adapter+ViewHolder
- **F4** — Fragment Transactions (no Activity restart)
- **F5** — Search / Filter on RecyclerView

↓ App Navigation Flow



2 Logic Map — Requirement Traceability Table

REQ. ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F1	`LoginActivity.kt`	`btnLogin.setOnClickListener{}`	Validates email + password fields. On success, creates an explicit `Intent(th` `MainActivity::class.java)` and attaches three extras via `putExtra()` : `EXTRA_USERNAME` , `EXTRA_SESSION_TOKEN` , `EXTRA_GREETING` . Calls `finis` pop LoginActivity off the back-stack. No SharedPreferences or static variables **Intent Extras** **No SharedPrefs**
F1	`DashboardActivity.kt`	`onCreate() — intent.getStringExtra()`	Host activity reads `EXTRA_USERNAME` , `EXTRA_GREETING` , and `EXTRA_SESSIO` from the incoming Intent and displays them in `tv_dashboard_greeting` and `tv_dashboard_username` TextViews. Also forwards session extras when laun child activities (Notifications, MainActivity). **Data Receiving** **Session Forwarding**
F1	`MainActivity.kt`	`onCreate() — intent.getStringExtra()`	Receives the same session extras forwarded from LoginActivity. Constructs an DashboardActivity and re-attaches all three extras, completing the full F1 relay Login → Main → Dashboard. **Intent Chain**
F2	`Destination.kt`	`writeToParcel() / createFromParcel()`	`Destination` is a `data class` implementing `Parcelable` . Serialises all 8 name, region, description, rating, pricePerNight, category, iconEmoji) via `Parc` `CREATOR` companion deserialises them on the receiving end. Zero global stat involved. **Parcelable Model** **Bundle Safe**
F2	`DestinationDetailFragment.kt`	`newInstance() / arguments?.getParcelable()`	Factory method `newInstance(destination)` is the *only* way to construct this fragment. It creates a `Bundle` , calls `putParcelable(ARG_DESTINATION, destination)` , and assigns it to `fragment.arguments` . In `onViewCreated(` Destination is recovered via `arguments?.getParcelable(ARG_DESTINATION)` TextViews are bound in `bindDestination()` . **Bundle Args** **Factory Pattern**
F2	`DashboardActivity.kt`	`onDestinationSelected(destination)`	Implements `DestinationListFragment.OnDestinationSelectedListener` . card is tapped, calls `DestinationDetailFragment.newInstance(destinatio` package the object into a Bundle, then launches the detail fragment via a Frag Transaction. This is the bridge between F2 Bundle passing and F4 transaction. **Interface Callback** **Fragment Launch**
F3	`DestinationAdapter.kt`	`onCreateViewHolder() / onBindViewHolder()`	Custom `RecyclerView.Adapter` with inner `DestinationViewHolder` class. ViewHolder caches 6 view references (emoji, name, region, category, rating, p avoid repeated `findViewById` calls. Inflates `item_destination_card.xml` . Maintains two lists: `fullList` (source of truth) and `filteredList` (what RecyclerView displays). **Custom Adapter** **ViewHolder Pattern**
F3	`DestinationListFragment.kt`	`onViewCreated() — recyclerView setup`	Sets `recyclerView.layoutManager = LinearLayoutManager(requireConte` and assigns the DestinationAdapter. Seed data is 12 real Pakistani destination Lahore, Fairy Meadows, Karachi, Skardu, Mohenjo-daro, Nathia Gali, Deosai, Rohtas Fort, Gwadar, Kalash Valley). **RecyclerView Host** **12 Destinations**

REQ. ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F4	`DashboardActivity.kt`	`showTab() — supportFragmentManager.beginTransaction()`	Four bottom-nav buttons each call `showTab(tabIndex)` . This calls `supportFragmentManager.beginTransaction().replace(R.id.fragment_co fragment).commit()` . No Activity restart occurs — only the fragment content s Handles back-stack awareness via `backStackEntryCount` . Tabs: Explore (DestinationListFragment), Trips (BookingHistoryFragment), Saved (ExploreGridFragment), Profile (ProfileFragment). Fragment Replace · No Restart
F4	`DashboardActivity.kt`	`onDestinationSelected() — addToBackStack()`	When a destination card is clicked, the detail fragment is loaded with `setCustomAnimations(slide_in_left, fade_out...)` and `.addToBackStack("detail")` . The back button pops the fragment rather tha the activity, fulfilling the "without restarting the activity" constraint. Back Stack · Custom Animation
F5	`DestinationAdapter.kt`	`getFilter() — performFiltering() / publishResults()`	Implements `Filterable` . The inner `Filter` object runs `performFilterin` background thread — matches query against `name` , `region` , and `category` (case-insensitive). `publishResults()` runs on the main thread, swaps `filteredList` , and calls `notifyDataSetChanged()` . Filterable · Background Thread
F5	`DestinationListFragment.kt`	`etSearch.addTextChangedListener{}`	Wires the search `EditText` (`et_search_destinations`) to the adapter's filt keystroke fires `adapter.filter.filter(text)` , providing real-time live searc any debounce delay. Clearing the field restores the full list instantly. Live Search · Keystroke Listener

3 Global Constraints — Compliance Evidence

🔒 Constraint 1 — Data Passing via Intent Extras & Bundles Only

Rule: No SharedPreferences, global static variables, or Singletons may be used for inter-screen communication.

Evidence: All session data flows exclusively through `Intent.putExtra()` / `intent.getStringExtra()`. The chain is: `LoginActivity` → `putExtra(EXTRA_USERNAME, EXTRA_SESSION_TOKEN, EXTRA_GREETING)` → `MainActivity` → `DashboardActivity` → `NotificationsActivity` etc. The `SessionIntent.kt` helper (`forwardSession()`) propagates session data between activities without any static field. The `Destination` model uses `Parcelable` + `Bundle` for inter-fragment transfer. No single `SharedPreferences.edit()` call exists in the entire codebase.

🧩 Constraint 2 — Modular UI via Fragments (Activities as Containers Only)

Rule: Primary UI content must live in Fragments. Activities are navigation coordinators.

Evidence: `DashboardActivity` contains a single `fragment_container` FrameLayout. All tab content is rendered by 4 Fragment classes: `DestinationListFragment`, `BookingHistoryFragment`, `ExploreGridFragment`, `ProfileFragment`. Destination detail is `DestinationDetailFragment`. The activity itself contains no list data or UI logic — it only calls `supportFragmentManager.beginTransaction()`.

📋 Constraint 3 — Dynamic Lists via RecyclerView + Custom Adapter + ViewHolder

Rule: All lists must use RecyclerView with a custom Adapter and ViewHolder.

Evidence: `DestinationAdapter` extends `RecyclerView.Adapter<DestinationAdapter.DestinationViewHolder>`. The inner `DestinationViewHolder` caches view references for emoji, name, region, category, rating, and price TextViews. `onCreateViewHolder()` inflates `item_destination_card.xml`. `onBindViewHolder()` calls `holder.bind(filteredList[position])`.

📁 Constraint 4 — Clean Architecture (Proper Folder / Package Organization)

Rule: Code must be organized into proper packages: Activities, Fragments, Adapters, Models.

Evidence: The project (`com.example.roamist`) is split into four sub-packages: `.activities` (16 classes), `.fragments` (5 classes), `.adapters` (1 class), `.models` (1 class). XML layouts are in `res/layout/` with naming conventions (`activity_*`, `fragment_*`, `item_*`). Separation of concerns is maintained — no business logic in XML, no UI binding in models.

⊕ Component Interaction Map

COMPONENT	TYPE	COMMUNICATES WITH	MECHANISM
`LoginActivity`	Activity	MainActivity, SignupActivity	Intent + putExtra()
`DashboardActivity`	Activity (Host)	All 5 Fragments, NotificationsActivity, MainActivity	FragmentTransaction + Intent Extras
`DestinationListFragment`	Fragment	DashboardActivity (via interface), DestinationAdapter	OnDestinationSelectedListener callback
`DestinationDetailFragment`	Fragment	Receives Destination from DashboardActivity	Bundle arguments (Parcelable)
`DestinationAdapter`	Adapter	DestinationListFragment (hosts RV), Destination model	onItemClick lambda callback + Filterable
`Destination`	Model	Adapter (binds data), DetailFragment (displays data)	Parcelable serialization

COMPONENT	TYPE	COMMUNICATES WITH	MECHANISM
`BookingHistoryFragment`	Fragment	DashboardActivity (swapped via showTab)	Fragment Transaction (.replace)
`ProfileFragment`	Fragment	DashboardActivity	Fragment Transaction (.replace)

4 GitHub Collaboration & Grading Rubric

GitHub Requirements

- All members added as Collaborators on repo
- Each member commits regularly with clear messages
- Pull latest before starting — merge conflicts resolved
- Commit messages describe major features / screens
- Evaluation via commit history & contribution graph

Recommended Commit Conventions

- **feat:** Add DestinationAdapter with Filterable [F3,F5]
- **feat:** Implement Intent navigation Login→Dashboard [F1]
- **feat:** Bundle Parcelable transfer to DetailFragment [F2]
- **feat:** Fragment transactions bottom nav tabs [F4]
- **fix:** Resolve search filter empty query case [F5]

★ Grading Rubric

EVALUATION CRITERIA	REQUIREMENTS COVERED	CLASSES INVOLVED	MARKS
Intent & Navigation Implementation	F1	LoginActivity → MainActivity → DashboardActivity	25
RecyclerView Implementation	F3	DestinationAdapter + DestinationViewHolder + item_destination_card.xml	25
Fragment Management	F4	DashboardActivity.showTab() + all 5 Fragment classes	20
Data Communication & Integrity	F2 F5	Destination (Parcelable) + DestinationDetailFragment.newInstance() + DestinationAdapter.getFilter()	20
GitHub Collaboration & Version Control	—	Repository commit history & contribution graph	10
TOTAL			**100**

Submission Checklist

DELIVERABLE	FORMAT	CONTENTS	STATUS
Android Studio Project	.ZIP	Complete Kotlin source, XML layouts, drawables, Gradle config	✓ Required
Logic Map Document	logic_map.pdf	This document — all 5 functional requirements traced to classes & methods	✓ This File
GitHub Repository	URL Link	Full project history, all members as collaborators, regular commits	✓ Required

📌 Key Takeaway — How the App Satisfies Every Constraint

No SharedPreferences used
All data travels via `Intent.putExtra()` and `Bundle.putParcelable()` exclusively.

RecyclerView + Adapter + ViewHolder
`DestinationAdapter` implements all three pillars with Filterable support for F5.

Fragments as primary UI containers
DashboardActivity is a pure host — all visible content lives inside the 5 Fragment classes.

Clean package structure
`.activities / .fragments / .adapters / .models` — each class lives in its correct package.